

Mathematics as a Programmable Language: Writing Mathematics in Go

Hyosang (Henry) Kang

Department of Liberal Arts and Sciences, DGIST, South Korea

Department of Infrastructure Engineering, University of Melbourne

Teaching Chat, 15 September 2026

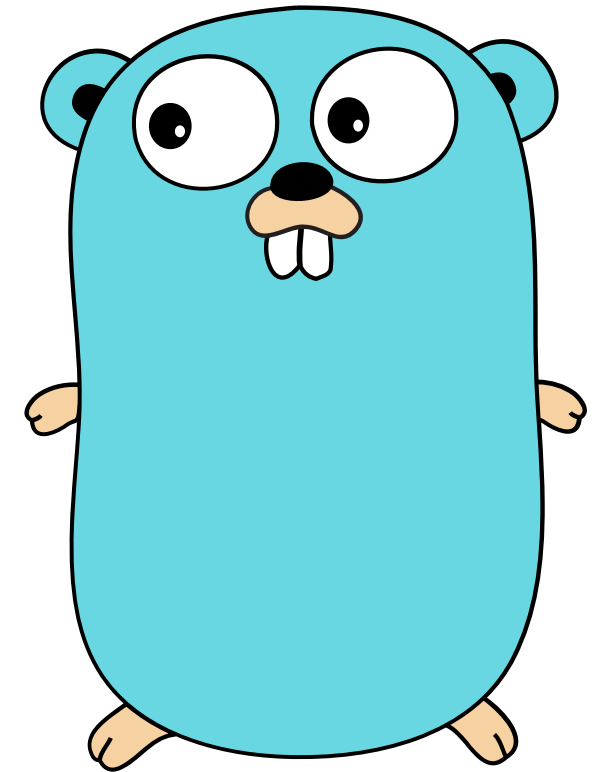
School of Mathematics and Statistics, University of Melbourne

Where the journey began

- Teaching first- and second-year mathematics subjects at DGIST
 - Multivariable Calculus, Linear Algebra, Engineering Mathematics I and II
 - Each class section runs independently
- Programming-based project assignments
 - Physics simulations, game development, and neural networks
 - MATLAB, Python, Go

Why Go?

- Created by Google engineers in 2007
 - Simplicity, strong typing, concurrency
 - **Interfaces** (abstract typing)
- Why not Python, Haskell, or Lean?
 - Python: less explicit type declarations
 - Haskell (or Lean): unfamiliar syntax
- Go in projects
 - class assignments
 - student research projects



A glimpse of Go: Computing $\sqrt{x}$ in Go

$$y_{n+1} = \frac{1}{2} \left(y_n + \frac{x}{y_n} \right)$$

```
func Sqrt(x float64) float64 {  
    var y float64 = 1  
    for i := 0; i < 30; i++ {  
        y = (y + x/y) / 2  
    }  
    return y  
}
```

A spectrum of mathematical statements

Example: continuity

Rigorous

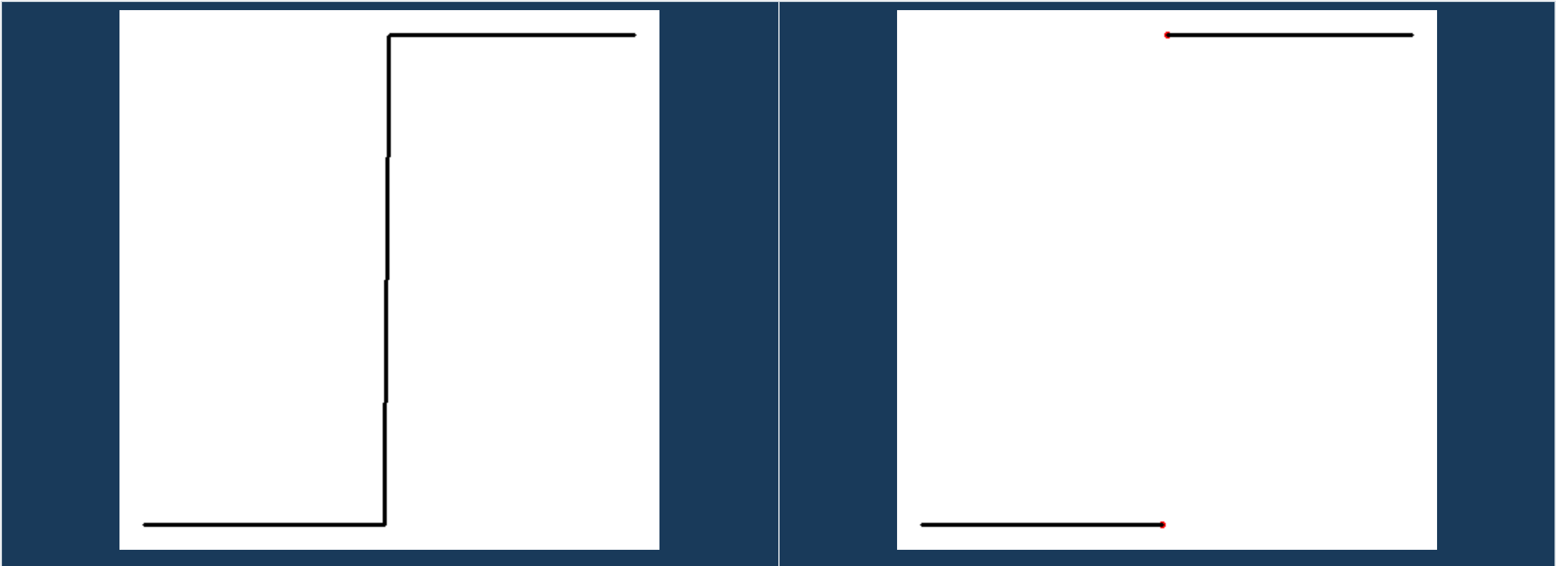
A function $y = f(x)$ is continuous at x_0 if, for every $\epsilon > 0$, there exists $\delta > 0$ such that $|f(x) - f(x_0)| < \epsilon$ whenever $|x - x_0| < \delta$.

Intuitive

A function is continuous if its graph has no breaks.

Objective: Giving students the freedom to express mathematical ideas with their own imagination and creativity.

Continuity is knowing when to break the graph



Warning: The following examples are what students **would** have done, not what they (should) have done.

Graph as a static type:

```
type Graph struct {  
    Points []Point  
    Lines  []Line  
}  
  
type Point struct {  
    X, Y float64  
}  
  
type Line struct {  
    From, To Point  
}
```

```
type Function struct {  
    Interval [2]float64  
    Formula  func(float64) float64  
}
```

Obtain a graph from a static type

```
const Nsub = 100

func (fn Function) Graph() Graph {
    a, b := fn.Interval[0], fn.Interval[1]
    xarr := Linspace(a, b, Nsub)
    var graph Graph
    for i := 0; i < Nsub; i++ {
        x0, x1 := xarr[i], xarr[i+1]
        y0, y1 := fn.Formula(x0), fn.Formula(x1)
        graph.Lines = append(graph.Lines, Line{
            From: Point{X: x0, Y: y0},
            To:   Point{X: x1, Y: y1},
        })
    }
    return graph
}
```

Drawing a discontinuous graph

```
func (fn Function) GraphContinuous(eps float64) Graph {
    graph := fn.Graph()
    var lines []Line
    var points []Point
    for _, l := range graph.Lines {
        if Abs(l.From.Y-l.To.Y) > eps {
            points = append(points, l.From, l.To)
        } else {
            lines = append(lines, l)
        }
    }
    return Graph{
        Points: points,
        Lines:  lines,
    }
}
```

Abstraction in mathematics and Go

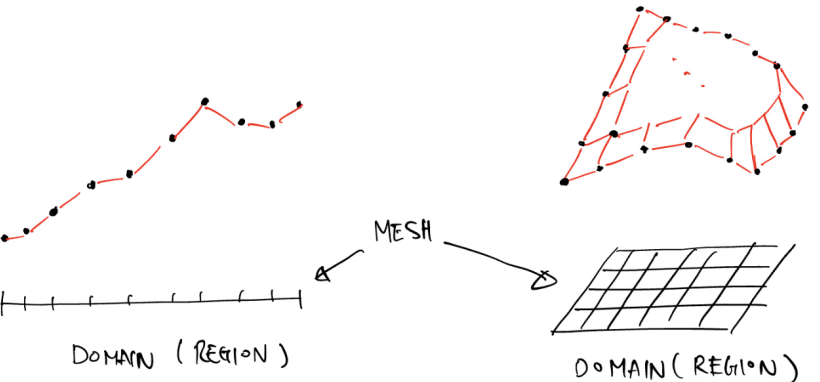
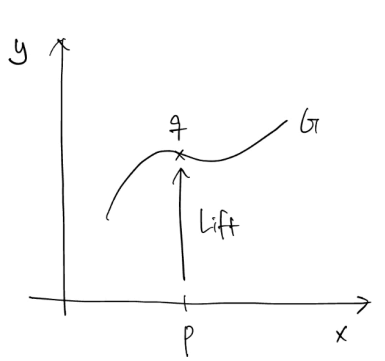
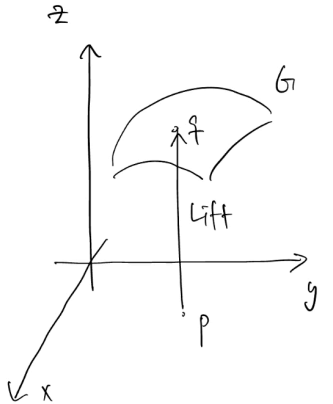
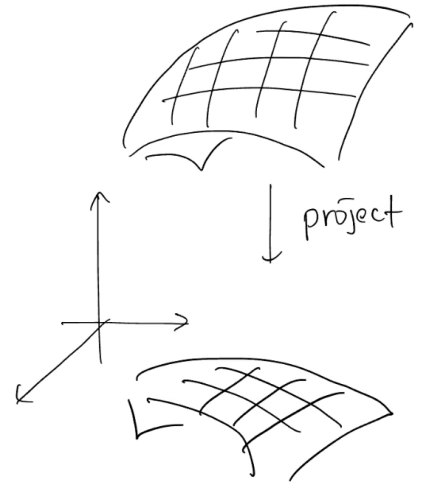
Mathematical objects are defined by their functionality.

An abstract type in Go is defined by its methods.



Graph as an abstract type:

```
type Grapher interface {  
    Region  
    Lift(Point) Point  
    Project(Point) Point  
}
```

Region	Lift	Project
	 	

```

type Grapher interface {
    Region
    Lift(Point) Point
    Project(Point) Point
}

```

Region

```

type Region interface {
    Set[Point]
    Corner() Vector
    Lengths() []Real
}

type Set[T Element] interface {
    Contains(T) bool
}

```

Vector

```

type Vector interface {
    Point
    Additive
    Scale(Real) Vector
    Inner(Vector) Real
    Basis() []Vector
}

type Real interface {
    Element
    Additive
    Multiplicative
    GreaterThan(Real) bool
    ToFloat() float64
}

```

Point

```

type Point interface {
    Element
    Finite
    Map[int, Real]
}

type Element interface {
    Equals(any) bool
}

type Finite interface {
    Len() int
}

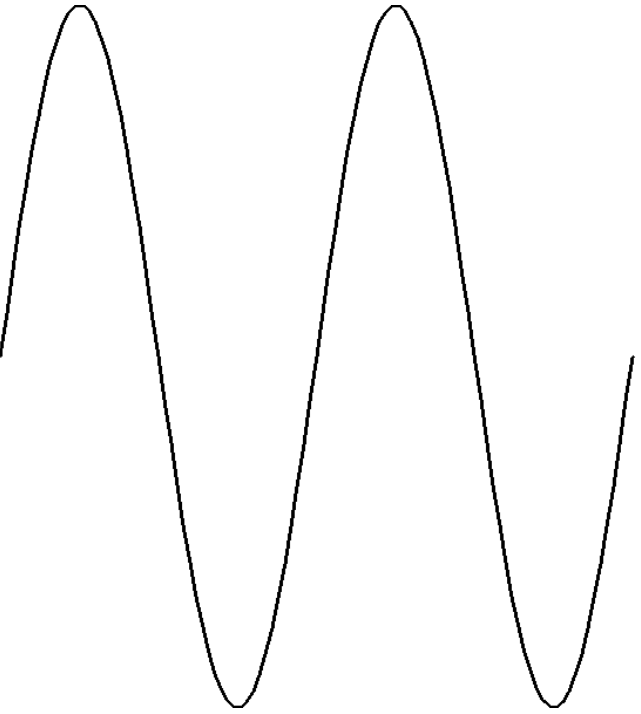
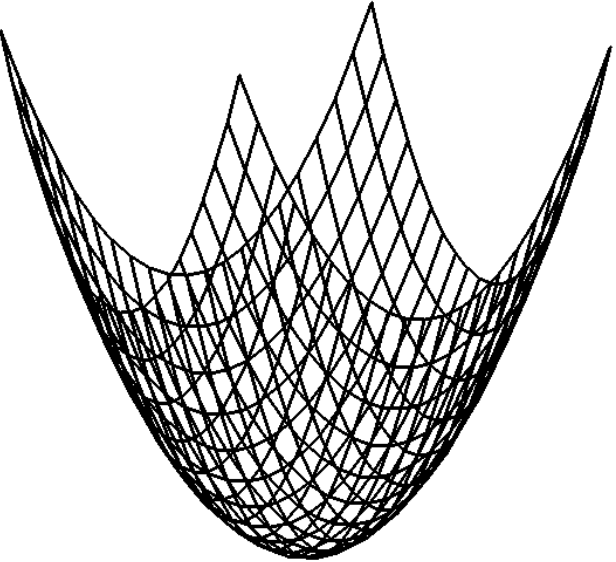
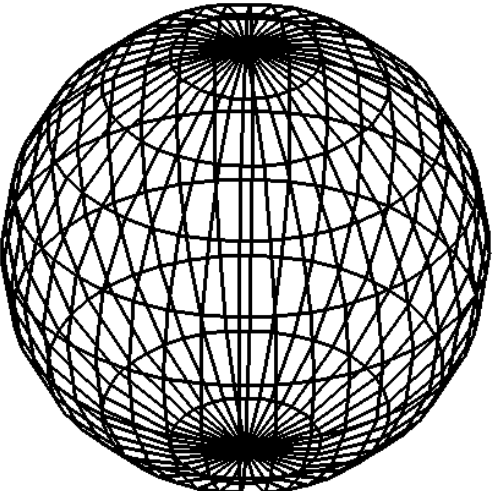
type Map[T1, T2 any] interface {
    Map(T1) T2
}

```

Drawing a graph from a Grapher

```
func Graph(gpr Grapher, nsub int) <-chan [2]Point {
    dim := gpr.Corner().Len()
    mesh := Mesh(gpr, nsub)
    ch := make(chan [2]Point)
    go func() {
        defer close(ch)
        for i, p := range mesh {
            for _, j := range Neighbors(i, nsub, dim) {
                q := mesh[j]
                ch <- [2]Point{gpr.Lift(p), gpr.Lift(q)}
            }
        }
    }()
    return ch
}
```

One algorithm works for all types

Single-variable	Two-variable	Parametric
		

Grapher type: single-variable function

```
type Single struct {  
    Interval  
    Eval func(float64) float64  
}  
  
func (s Single) Lift(p gocalc.Point) gocalc.Point {  
    x := p.Map(0).ToFloat()  
    return &SimpleVector{  
        x,  
        s.Eval(x),  
    }  
}  
  
func (s Single) Project(p gocalc.Point) gocalc.Point {  
    return p  
}
```

Grapher type: two-variable function

```
type Double struct {  
    Rect  
    Eval func(float64, float64) float64  
}  
  
func (d Double) Lift(p gocalc.Point) gocalc.Point {  
    x, y := p.Map(0).ToFloat(), p.Map(1).ToFloat()  
    return SimpleVector{x, y, d.Eval(x, y)}  
}  
  
func (d Double) Project(p gocalc.Point) gocalc.Point {  
    t1, t2 := -1.0, -1.0  
    x, y, z := p.Map(0).ToFloat(), p.Map(1).ToFloat(), p.Map(2).ToFloat()  
    x, y = x*math.Cos(t1)-y*math.Sin(t1), x*math.Sin(t1)+y*math.Cos(t1)  
    y, _ = y*math.Cos(t2)-z*math.Sin(t2), y*math.Sin(t2)+z*math.Cos(t2)  
    return SimpleVector{x, y}  
}
```

Drawing a discontinuity

```
type Continuous interface {
    Map[Point, Vector]
    Eps(Real, Point) Real
}

func Graph(gpr Grapher, nsub int) <-chan [2]Point {
    ...
    if fn, ok := gpr.(Continuous); ok {
        delta := Distance(p, q)
        eps := fn.Eps(delta, p)
        if eps.GreaterThan(Distance(fn.Map(p), fn.Map(q))) {
            ch <- line
        }
    } else {
        ch <- line
    }
    ...
}
```

This is **not** about

Making computation more
efficient:

```
func Sqrt(a Real) Real {  
    x := a.One().(Real)  
    half := Rational(x, [2]int{1, 2})  
    for !a.Equals(x.Mul(x).(Real)) {  
        q := a.Mul(x.MulInv()).(Real)  
        x = x.Add(q).(Real)  
        x = x.Mul(half).(Real)  
    }  
    return x  
}
```

Translating every mathematical
definition into code:

```
type Set[T Element] interface {  
    Contains(T) bool  
}  
  
type Additive interface {  
    Add(Additive) Additive  
    AddInv() Additive  
    Zero() Additive  
}
```

Interfaces express operations, not laws

Real numbers

```
type RealNumber interface {  
  
    Zero() RealNumber  
    AddInv() RealNumber  
    Add(RealNumber) RealNumber  
  
    One() RealNumber  
    MulInv() RealNumber  
    Mul(RealNumber) RealNumber  
  
    LessThan(RealNumber) bool  
  
    Evaluation() float64  
  
}
```

Natural numbers

```
type Nat interface {  
    Back() Nat  
    Next() Nat  
}  
  
func Iter(n Nat) chan Nat {  
    ch := make(chan Nat)  
    go func() {  
        i := Zero(n).(Nat)  
        for n.Back() != nil {  
            ch <- i  
            i = i.Next()  
            n = n.Back()  
        }  
        close(ch)  
    }()  
    return ch  
}
```

About (Korean) students at DGIIST

- Many have studied in a highly competitive educational environment with a focus on:
 - The standardised test (Suneung)
 - High school grades
 - School activities tailored to university admissions
- Many studied calculus in high school
 - Strong emphasis on calculation
 - Limited emphasis on rigorous reasoning
 - Varied perceptions of mathematics
 - Some enjoy mathematics, but in a narrow sense.
 - Some dislike mathematics because it feels less dynamic.

Where to?

- Scope
 - How easy can this be?
 - How deep can this be?
- Students
 - How much should students know about Go?
 - Should it be more interactive (like games)?
- Class
 - How much of the regular curriculum can this cover?
 - Should it be used as an extra or a replacement?

**Students may not need
mathematical programming,
but they need
programming mathematics.**

Upcoming talk: **Make Games, Learn Math**

Date and time: Thursday 24 September, 10:00 am–11:00 am

Venue: Melbourne Connect, **Level 5, Danda-gat-ith** (**Free coffee! BYOM**)

